EXPERIENCIAS FORMATIVAS EN SITUACIONES REALES DE TRABAJO

(EFSRT II) — INFORME TÉCNICO Y ARQUITECTURA

MANUAL TÉCNICO Y ARQUITECTURA

SISTEMA WEB EMPRESARIAL POR CAPAS (N-TIER)

Caso de Estudio Empresarial:

DISTRIBUCIONES Y SERVICIOS DEL NORTE S.A.C. (PIURA)

DATOS DEL PROYECTO	DETALLE OFICIAL
Empresa Caso	Distribuciones y Servicios del Norte S.A.C. (RUC 20543210981)
Ubicación	Av. Sánchez Cerro Nro. 1250, Piura, Perú
Rubro	Comercialización y distribución mayorista a instituciones públicas
Equipo Desarrollador	Grupo N° 4 - Cibertec
Integrantes	• Castillo Sierra Sebastian • Milian Chipana Raul • Tucto Jurado Nicole Andrea • Terrones Gallardo Renzo
Docente Tutor	Jorge Luis Villavicencio Diaz
Stack Tecnológico	Frontend React 19 + Vite (Puerto 5190) | Backend Node.js Express (Puerto 4000) | SQL Server (Puerto 1433)
Estado de Entrega	Versión Final Evaluativa (Sustentación)

AVISO TÉCNICO DE PUERTOS Y ACCESIBILIDAD LOCAL

- URL Oficial del Frontend Web: http://localhost:5190/
- URL Oficial del Backend REST API: http://localhost:4000/api/
- Se configuró estrictamente el puerto 5190 en vite.config.js con strictPort: true para evitar cualquier interferencia o cruce con otros proyectos que corran en puertos por defecto (como 5173 o 5174).

1. Diagnóstico Situacional y Metas SMART

La empresa Distribuciones y Servicios del Norte S.A.C. gestionaba sus operaciones de venta institucional mediante hojas de cálculo descentralizadas (Excel). Esto generaba tres problemas críticos: descuadres frecuentes de inventario por falta de concurrencia, errores humanos en el despacho de productos y tiempos excesivos de espera para emitir pedidos a programas sociales (Vaso de Leche, Qali Warma).

Cumplimiento del Indicador Oficial de Desempeño

Para medir el impacto técnico del sistema web por capas, se aplicó la fórmula oficial del indicador de productividad:

```
Indicador SMART (I) = [(T_sin - T_con) / T_sin] * 100

• T_sin (Tiempo sin sistema): 15 minutos (900 segundos) de validación manual en Excel.
• T_con (Tiempo con sistema web relacional): 18 milisegundos (0.018 s) mediante API y Stored Procedure.
• Cálculo: I = [(900 - 0.018) / 900] * 100 = 99.998% de Ahorro de Tiempo de Atención.
```

El resultado supera con holgura la meta mínima del 86% establecida para la semana evaluativa, logrando un despacho inmediato y confiable.

Metas Cualitativas Alcanzadas:

- **Descuadres de Stock:** Reducción al 0% gracias a transacciones atómicas ACID y bloqueo por fila con UPDLOCK.
- **Trazabilidad Total:**Base de datos en 3FN con historial de pedidos, detalle unitario y estados auditables.
- **Concurrencia Segura:** Control de concurrencia nativo en SQL Server para múltiples operadores en ventanilla.

2. Arquitectura del Sistema Web en 3 Capas (N-Tier)

El proyecto implementa una Arquitectura Limpia Desacoplada (Clean Architecture) que divide las responsabilidades en tres capas independientes:

CAPA	TECNOLOGÍA	RESPONSABILIDADES Y PATRONES
1. Presentación (Frontend)	React 19, Vite (Puerto 5190)	• Dashboard interactivo con velocímetro SMART e indicadores en tiempo real. • Simulador transaccional del Stored Procedure (escenarios Éxito y Stock Insuficiente). • Reportes con exportación a PDF corporativo y Excel (.xlsx) estructurado. • Módulos CRUD de clientes institucionales e inventario en almacén.
2. Lógica y Negocio (Backend)	Node.js 24, Express (Puerto 4000)	• Rutas RESTful semánticas (/api/pedidos, /api/productos, /api/clientes, /api/reportes). • Capa de Servicios con benchmarking milimétrico (perf_hooks). • Capa de Repositorios con consultas parametrizadas (anti SQL Injection). • Connection Pool de SQL Server con fallback transparente en memoria.
3. Persistencia (Datos)	Microsoft SQL Server (Puerto 1433)	• Esquema normalizado en Tercera Forma Normal (3FN). • Stored Procedure transaccional sp_ProcesarPedidoInstitucional con UPDLOCK. • Stored Procedure sp_ReporteConsolidadoInventario para agregación de stock. • Disparador (Trigger) trg_ValidarDetallePedido para integridad relacional.

3. Análisis Detallado de la Base de Datos (SQL Server)

El esquema de base de datos fue optimizado a partir de las necesidades reales de Distribuciones y Servicios del Norte S.A.C., corrigiendo redundancias y asegurando atomicidad.

A. Normalización en 3FN (Tercera Forma Normal)

- **1FN (Primera Forma):** Atributos indivisibles y claves primarias IDENTITY(1,1). No existen listas concatenadas.
- **2FN (Segunda Forma):** Cada columna no clave depende en su totalidad de la clave primaria (sin dependencias parciales).
- **3FN (Tercera Forma):** No hay dependencias transitivas. La categoría de un producto se extrae mediante la clave foránea IdCategoria hacia la tabla maestra Categoria.

B. Optimización de Rendimiento: Índices Non-Clustered B-Tree

Para evitar escaneos completos de tablas (Table Scans) ante un crecimiento a decenas de miles de registros, se agregaron índices secundarios:

```
CREATE NONCLUSTERED INDEX IX_Producto_Categoria ON Producto(IdCategoria);
CREATE NONCLUSTERED INDEX IX_Pedido_Cliente ON Pedido(IdCliente);
CREATE NONCLUSTERED INDEX IX_DetallePedido_Pedido ON DetallePedido(IdPedido);
CREATE NONCLUSTERED INDEX IX_DetallePedido_Producto ON DetallePedido(IdProducto);
```

C. Algoritmo Transaccional ACID: sp_ProcesarPedidoInstitucional

Este procedimiento almacenado encapsula la operación crítica del negocio con manejo de excepciones (TRY...CATCH) y bloqueo pesimista mediante la directiva WITH (UPDLOCK):

```
CREATE PROCEDURE sp_ProcesarPedidoInstitucional
    @IdCliente INT, @IdProducto INT, @CantidadSolicitada INT,
    @Resultado INT OUTPUT, @Mensaje VARCHAR(200) OUTPUT
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- Bloqueo pesimista de fila para evitar condiciones de carrera (Race Conditions):
        SELECT @StockActual = StockActual, @PrecioU = PrecioUnitario
        FROM Producto WITH (UPDLOCK) WHERE IdProducto = @IdProducto;

        IF @StockActual < @CantidadSolicitada BEGIN
            ROLLBACK TRANSACTION;
            SET @Resultado = 0; SET @Mensaje = 'Stock insuficiente para abastecer el pedido.';
            RETURN;
        END;
        -- Inserción de cabecera de Pedido, Detalle y Descuento de Stock en Producto...
        COMMIT TRANSACTION;
        SET @Resultado = 1; SET @Mensaje = 'Pedido procesado con éxito.';
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        SET @Resultado = -1; SET @Mensaje = ERROR_MESSAGE();
    END CATCH
END;
```

PROPIEDADES ACID DEMOSTRADAS EN EL PROCEDIMIENTO

- Atomicidad: Inserción de pedido y descuento de inventario se realizan juntos o se anulan con ROLLBACK.
- Consistencia: Ninguna regla de negocio o restricción CHECK(StockActual >= 0) puede ser violada.
- Aislamiento: WITH (UPDLOCK) previene lecturas sucias y actualizaciones perdidas entre operadores.
- Durabilidad: Tras el COMMIT, SQL Server almacena las operaciones en el Transaction Log (.ldf).

4. Análisis del Backend (Node.js & Express)

El backend actúa como intermediario seguro entre la interfaz React y la base de datos SQL Server, aplicando el principio de mínima exposición y desacoplamiento:

- **Connection Pool:** Mantiene un conjunto reciclable de hasta 15 conexiones TCP abiertas con MSSQL.
- **Mecanismo de Resiliencia:** Si SQL Server no está iniciado al momento de levantar el servidor, conmuta automáticamente a un almacén en memoria sin lanzar un error fatal 500.
- **Capa Repositorio (DAL):** Previene la inyección SQL mediante el uso mandatorio de parámetros tipados (request.input(sql.Int)).
- **Capa Servicio (BLL):** Implementa la medición de latencia y cálculo del indicador de desempeño con performance.now().
- **Capa Controladores:** Valida los datos de entrada HTTP y entrega códigos estándar: 200, 201, 400, 404 y 500.

Catálogo de Endpoints RESTful

MÉTODO	ENDPOINT	DESCRIPCIÓN Y CAPA DE DATOS
GET	**/api/health**	Healthcheck de estado del servidor y conexión a base de datos
GET	**/api/productos**	Listado general de catálogo de productos con stock y precio
GET	**/api/clientes**	Listado de clientes e instituciones públicas registradas
POST	**/api/clientes**	Registro de nuevo cliente institucional con validación de RUC/DNI
GET	**/api/pedidos**	Historial detallado de pedidos con desglose de ítems y totales
POST	**/api/pedidos/procesar**	Ejecución transaccional de sp_ProcesarPedidoInstitucional
GET	**/api/reportes/consolidado**	Ejecución de sp_ReporteConsolidadoInventario con agregación
GET	**/api/reportes/kpis**	Cálculo de valor total de inventario, stock crítico y métricas SMART

5. Análisis del Frontend (React + Vite)

Construido con React 19 y Vite bajo una experiencia de usuario (UX) corporativa moderna, con diseño responsivo, gráficos KPI y soporte de exportación profesional:

- **DashboardView:** Visualización rápida de métricas clave, estado de inventario y medidor SMART.
- **ProcesarPedidoView:** Simulador del Stored Procedure que permite ejecutar pedidos y ver en pantalla si el resultado fue un COMMIT verde o un ROLLBACK rojo por falta de stock.
- **ReportesExportablesView:** Permite filtrar el inventario consolidado y generar reportes en Excel (.xlsx) mediante SheetJS y en PDF corporativo mediante jsPDF y jspdf-autotable.
- **ClientesView & InventarioView:** Gestión de datos maestros de clientes y existencias en almacén.

6. Configuración de Red y Prevención de Cruce de Puertos

Para responder puntualmente a la observación del usuario sobre el enlace http://localhost:5174/ (el cual correspondía a otro proyecto abierto en el equipo), se implementaron las siguientes configuraciones de aislamiento:

DIAGNÓSTICO: ¿POR QUÉ APARECIÓ EL PUERTO 5174?

Por defecto, Vite inicia en el puerto 5173. Si encuentra dicho puerto ocupado por otra aplicación, Vite salta automáticamente al puerto 5174, 5175, etc. Esto provocaba que si el usuario tenía otro proyecto en ejecución, los enlaces podían cruzarse.

SOLUCIÓN IMPLEMENTADA:
1. Se configuró un puerto dedicado y exclusivo (Puerto 5190) en vite.config.js.
2. Se activó strictPort: true, lo que impide que Vite salte a otro puerto si 5190 estuviera ocupado.
3. El frontend de EFSRT II corre de manera única y garantizada en: http://localhost:5190/

Configuración en vite.config.js:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5190,        // Puerto exclusivo para EFSRT II
    strictPort: true,  // Si está ocupado, no cambia a otro puerto aleatorio
    host: true         // Permite acceso en red local
  }
});
```

Resumen de Puertos Asignados al Proyecto:

COMPONENTE	PUERTO	URL DE ACCESO	ESTADO
Frontend (Vite + React)	5190	http://localhost:5190/	Dedicado y Bloqueado (strictPort: true)
Backend (API REST)	4000	http://localhost:4000/api/	Dedicado (Configurable en .env)
Base de Datos (SQL Server)	1433	localhost:1433	Instancia SQL Server Local o Remota

Comandos Oficiales de Ejecución:

Para levantar ambos servicios en terminales independientes de PowerShell:

```
# Terminal 1 - Iniciar Backend (Puerto 4000):
cd "e:\Cursos\Cibertec\Ciclo 3\EFSRTII\Proyecto\backend"
npm run dev

# Terminal 2 - Iniciar Frontend (Puerto 5190):
cd "e:\Cursos\Cibertec\Ciclo 3\EFSRTII\Proyecto\frontend"
npm run dev
```

7. Banco de Preguntas Clave para la Sustentación ante el Jurado

Preguntas frecuentes y respuestas técnicas fundamentadas para la evaluación final del curso EFSRT II:

P: ¿Por qué se eligió una arquitectura en 3 capas en lugar de un monolito simple?

R: Porque desacopla responsabilidades según el principio de Single Responsibility (SRP). La interfaz de React se enfoca en la usabilidad y exportación sin conocer credenciales ni detalles de SQL; el backend Express aplica validaciones y reglas de negocio; y SQL Server asegura la integridad referencial y atomicidad. Esto permite que el sistema sea escalable, seguro y fácil de mantener.

P: ¿Cómo se demostró el cumplimiento de las propiedades ACID en la base de datos?

R: A través del Stored Procedure sp_ProcesarPedidoInstitucional. Atomicidad: se hace COMMIT de todo el pedido o ROLLBACK total si falla el stock. Consistencia: las restricciones CHECK impiden existencias negativas. Aislamiento: la directiva WITH (UPDLOCK) bloquea la fila del producto evitando que dos operarios vendan el mismo stock concurrentemente. Durabilidad: el motor escribe los cambios en el archivo .ldf antes de confirmar la transacción.

P: ¿Qué diferencia existe entre un INNER JOIN y un LEFT JOIN en el reporte consolidado de inventario?

R: En sp_ReporteConsolidadoInventario se utilizó LEFT JOIN con DetallePedido para no omitir aquellos productos nuevos que aún no registran ventas. Si se hubiera utilizado INNER JOIN, los productos sin ventas desaparecerían del inventario general. Con LEFT JOIN e ISNULL(SUM(dp.Cantidad), 0), dichos productos se muestran correctamente con 0 unidades vendidas.

P: ¿Cómo se garantiza la seguridad contra inyecciones SQL en el Backend?

R: En la capa de repositorios nunca se concatenan cadenas de texto para armar sentencias SQL. Se utiliza el objeto request.input() de la librería mssql, el cual envía los valores al motor tipados como parámetros binarios independientes, neutralizando cualquier intento de inyección.

P: ¿Cómo se integraron los Objetivos SMART en la aplicación?

R: El sistema incluye un módulo en el backend que calcula con milisegundos de precisión el tiempo de respuesta y lo contrasta con los 15 minutos que demoraba el proceso manual, arrojando un ahorro superior al 99% que se visualiza en un velocímetro interactivo en el Dashboard.